Tutorial - 6 23/2/2026

Name: Varadraj Patil
Roll no: 65
Prn : 12412485
Class : CSSE-B

Problem Statement : Prepare effective test cases using suitable black-box and white-box testing techniques for a given software module.

Solution : Preparing Effective test cases using black-box and white-box testing techniques

1. Introduction

→ Software testing ensures that a software system works as expected and is free from defects. Test cases are systematically designed scenarios used to verify functionality, performance, and correctness.

→ Testing techniques are broadly classified into :

- Black-box testing - tests functionality without looking at internal code
- White-box testing - tests internal logic, structure, and code paths.

→ This tutorial demonstrates both techniques using a suitable software module.

→ Black box testing

- Black box testing is a software testing technique in which the tester verifies the

29

functionality of an application with
owing the internal code, structure
implementation.
The tester focuses only on: inputs
outputs, functional requirements
It is also called: behavioral tes
functional testing.

- Example
→ Suppose you are testing a login f
Input: username → admin, password
Expected output: login successful
→ You do not check how the password
validated internally
→ You only check whether the correct
displayed.
- Common ~~bla~~ black-box techniques
i) Equivalence partitioning
ii) Boundary value analysis
iii) Decision table testing
iv) State transition testing
v) Use case testing
- Advantages
i) No programming language required
ii) Tests real user behaviour
iii) Good for functional validation
- Disadvantages
i) Internal code errors may not be detect
ii) Limited coverage of internal paths
→ White box testing
- It is a software testing technique
the tester

and code of the application.
It is also called:
. Structural testing.
. Glass-box testing.
. Clear-box testing
- Example:

```
if password == "1234":
    print ("login Successful")
else:
    print ("Invalid Password")
```

- In white-box testing you:
. Check both True and False conditions
. Ensure all lines of code execute
. Test all logical branches.
- Common White-box techniques
i) Statement coverage
ii) Brach coverage
iii) Path coverage
iv) Loop testing
v) Condition coverage
- Advantages.
i) Detects logical errors
ii) Ensures full code coverage
iii) Improves code quality
- Disadvantages
i) Requires programming knowledge
ii) Time-consuming.

(Q2)

2. Software Module selected
⇒ Example module : Personal Expense tra
management module.
Module name : Expense Manager
→ Functional description :
The module allows users to manage
expenses by :
. Adding new expenses
. Viewing all recorded expenses
. Setting and monitoring a budget
. Calculating total and remaining balan
. Saving / loading data from files
→ Business rules :
i) Expense amount must be greater th
ii) Category, date and description must
iii) Budget must be greater than or equ
iv) Total expenses are calculated as the
all entered expenses
v) Remaining balance = Budget - total
vi) If total expenses exceed budget → dis
vii) Expenses should be stored and retrie
file correctly.

3. Black-box testing techniques
→ Black-box testing focuses on user i
expected outputs without analyzing int
3.1 Equivalence partitioning (EP)
Input parameter : Expense amount

Partition type	Condition	Exam
Valid	>0	500
Invalid	≤ 0	-50
Invalid		0

1. Test cases using EP

TC ID	Input amount	Expected result
TC01	500	Expense added successfully
TC02	-50	Error: Invalid amount
TC03	0	Error: Invalid amount

2 Boundary value analysis (BVA)
Boundary around 0

TC ID	Amount	Expected result
TC04	1	Success
TC05	0	Error
TC06	-1	Error

3 Decision table testing
Conditions:
- Valid amount?
- Budget set?
- total > budget?

Amount valid	Budget set	Exceeds budget	Expected output
Yes	Yes	No	Expense added
Yes	Yes	Yes	Alert shown
No	Yes	-	Error
Yes	No	-	Expense added (no alert)

4 State Transition testing
States:
- No expenses
- Expense added
- Budget set
- Budget exceeded

2)

Current state	Action	Next state
No expenses	Add expense	Expense add
Expense added	Set budget	budget set
budget set	Add expense	Budget exceed
		(if limit

4. White-box testing techniques
→ White-box testing checks internal logic program

4.1 Sample Pseudocode

```
function addExpense (amount, budget, totalExp
    if amount <=0:
        return "Invalid amount"

    totalExpense += amount

    if budget >0 and totalExpense > budget
        return "Budget exceeded alert"

    return "Expense added successfully.
```

4.2 Statement coverage

Condition	True tested	False tested
amount <=0	✓	✓
budget >0	✓	✓
total > budget	✓	✓

4.3 Path coverage
Independent paths :
1) Invalid amount
2) Valid expense (no budget)

Valid expense (within budget)
budget exceeded
Minimum 4 test cases required

Cyclomatic complexity
Decision nodes :
amount <=0
budget > 0
total > budget
Cyclomatic complexity = 3+1 = 4
So, minimum 4 independent test paths.

Complete test case template.

Test case ID	Description	Input	Expected output	type
TC01	Valid expense	amount=500	expense added	black-box
TC02	Invalid amount	amount=-50	error	black-box
TC03	No budget	budget=0	expense added	white-box
TC04	budget exceeded	total>budget	Alert message	both
TC05	boundary test	amount=0	error	black-box